

Dominando los SQL JOINS:

Guía de Gestión Universitaria

Fundamentos y Caso de Uso

¿Qué es un SQL JOIN?

Operación que combina registros de dos tablas (A y B) mediante una clave común, permitiendo responder preguntas complejas que una sola tabla no puede resolver.



Tablas de Referencia (Contexto Universitario)

- Estudiantes (ID, nombre)
- Cursos (ID, nombre_curso)
- Requisitos_Previos (ID_curso, ID_requisito)



Los 6 Tipos de JOIN

1. INNER JOIN

Intersección Exacta

Devuelve solo estudiantes que están inscritos exactamente en algún curso. Si un estudiante no tiene curso, desaparece del resultado.



```
SELECT * FROM Estudiantes
JOIN Cursos
ON Estudiantes.id_curso = Cursos.id_curso;
```

2. LEFT JOIN (Outer)

Preservar la Izquierda

Muestra todos los estudiantes. Si un estudiante no está inscrito en ningún curso, el campo del curso aparecerá como NULL.

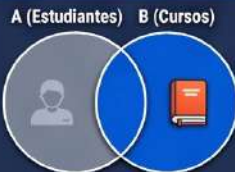


```
SELECT * FROM Estudiantes
LEFT JOIN Cursos
ON Estudiantes.id_curso = Cursos.id_curso;
```

3. RIGHT JOIN (Outer)

Preservar la Derecha

Muestra todos los cursos ofertados. Si un curso no tiene estudiantes inscritos, los campos del estudiante aparecerán como NULL.



```
SELECT * FROM Estudiantes
RIGHT JOIN Cursos
ON Estudiantes.id_curso = Cursos.id_curso;
```

4. FULL OUTER JOIN

Unión Simétrica Total

Combina todos los estudiantes y todos los cursos, sin importar si tienen pareja o no. Útil para auditoría de datos.



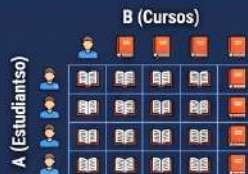
```
SELECT * FROM Estudiantes
FULL OUTER JOIN Cursos
ON Estudiantes.id_curso = Cursos.id_curso;
```

Nota de Emulación: En HIERGL/SQL es como si en LEFT JOIN y en RIGHT JOIN iniciaras el conector MINER.

5. CROSS JOIN

Producto Cartesiano

Genera todas las combinaciones posibles. Cruza cada estudiante con cada curso ofertado (útil para generar matrices de horarios vacías).

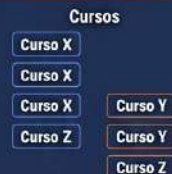


```
SELECT * FROM Estudiantes
CROSS JOIN Cursos;
```

6. SELF JOIN

Jerarquía de Cursos

Una la tabla Cursos consigo misma para mapear qué curso es requisito de otro usando alias (c1, c2).



```
SELECT c1.nombre AS Curso, c2.nombre AS Requisito
FROM Cursos c1 JOIN Cursos c2
ON c1.id_requisito = c2.id_curso;
```

Tabla de Referencia Rápida

Tipo de JOIN	Registros de A (Estudiantes)	Registros de B (Cursos)	Resultado Típico
INNER	Solo con pareja	Solo con pareja	Intersección estricta
LEFT	Todos	Solo con pareja	Lista total de alumnos
RIGHT	Solo con pareja	Todos	Oferta académica completa
FULL	Todos	Todos	Reconciliación total
CROSS	Combinación total	Combinación total	Matriz de datos

Conclusión y Fuentes

Importancia

Elegir el JOIN correcto garantiza la integridad de los reportes y optimiza el rendimiento de la base de datos al evitar procesar datos innecesarios.

Fuentes Consultadas

SQL Practico Online, Atlassian.